

Send and receive messages from Azure Queue storage

In this exercise, you create and configure Azure Queue Storage resources, then build a .NET app to send and receive messages using the **Azure.Storage.Queues** SDK. You learn how to provision storage resources, manage queue messages, and clean up your environment when finished.

Tasks performed in this exercise:

- Create Azure Queue storage resources
- Assign a role to your Microsoft Entra user name
- Create a .NET console app to send and receive messages
- Clean up resources

This exercise takes approximately **30** minutes to complete.

Create Azure Queue storage resources

In this section of the exercise you create the needed resources in Azure with the Azure CLI.

1. In your browser navigate to the Azure portal <https://portal.azure.com>; signing in with your Azure credentials if prompted.
2. Use the [**>**_] button to the right of the search bar at the top of the page to create a new cloud shell in the Azure portal, selecting a **Bash** environment. The cloud shell provides a command line interface in a pane at the bottom of the Azure portal. If you are prompted to select a storage account to persist your files, select **No storage account required**, your subscription, and then select **Apply**.

Note: If you have previously created a cloud shell that uses a *PowerShell* environment, switch it to **Bash**.

3. In the cloud shell toolbar, in the **Settings** menu, select **Go to Classic version** (this is required to use the code editor).
4. Create a resource group for the resources needed for this exercise. If you already have a resource group you want to use, proceed to the next step. Replace **myResourceGroup** with a name you want to use for the resource group. You can replace **eastus** with a region near you if needed.

In this **Cloudslice** lab, your **Resource Group** has already been created (**myResourceGrouplod58071234**) and you may safely skip this step.

TypeCopy

```
az group create --name myResourceGroup --location eastus
```

5. Many of the commands require unique names and use the same parameters. Creating some variables will reduce the changes needed to the commands that create resources. Run the following commands to create the needed variables. Replace **myResourceGroup** with the name you're using for this exercise. If you changed the location in the previous step, make the same change in the **location** variable.

TypeCopy

```
resourceGroup=myResourceGrouplod58071234
```

```
location=eastus
```

```
storAcctName=storactname58071234
```

6. You will need the name assigned to the storage account later in this exercise. Run the following command and record output.

TypeCopy

```
echo $storAcctName
```

7. Run the following command to create a storage account using the variable you created earlier. The operation takes a few minutes to complete.

bashTypeCopy

```
az storage account create --resource-group $resourceGroup \
```

```
--name $storAcctName --location $location --sku Standard_LRS
```

Assign a role to your Microsoft Entra user name

To allow your app to send and receive messages, assign your Microsoft Entra user to the **Storage Queue Data Contributor** role. This gives your user account permission to create queues, and send/receive messages using Azure RBAC. Perform the following steps in the cloud shell.

1. Run the following command to retrieve the **userPrincipalName** from your account. This represents who the role will be assigned to.

TypeCopy

```
userPrincipal=$(az rest --method GET --url https://graph.microsoft.com/v1.0/me \
```

```
--headers 'Content-Type=application/json' \
```

```
--query userPrincipalName --output tsv)
```

2. Run the following command to retrieve the resource ID of the storage account. The resource ID sets the scope for the role assignment to a specific namespace.

TypeCopy

```
resourceID=$(az storage account show --resource-group $resourceGroup \
--name $storAcctName --query id --output tsv)
```

3. Run the following command to create and assign the **Storage Queue Data Contributor** role.

TypeCopy

```
az role assignment create --assignee $userPrincipal \
--role "Storage Queue Data Contributor" \
--scope $resourceID
```

Create a .NET console app to send and receive messages

Now that the needed resources are deployed to Azure the next step is to set up the console application. The following steps are performed in the cloud shell.

Tip: Resize the cloud shell to display more information, and code, by dragging the top border. You can also use the minimize and maximize buttons to switch between the cloud shell and the main portal interface.

1. Run the following commands to create a directory to contain the project and change into the project directory.

TypeCopy

```
mkdir queuestor
cd queuestor
```

2. Create the .NET console application.

TypeCopy

```
dotnet new console
```

3. Run the following commands to add the **Azure.Storage.Queues** and **Azure.Identity** packages to the project.

TypeCopy

```
dotnet add package Azure.Storage.Queues
dotnet add package Azure.Identity
```

Add the starter code for the project

1. Run the following command in the cloud shell to begin editing the application.

TypeCopy

code Program.cs

2. Replace any existing contents with the following code. Be sure to review the comments in the code, and replace with the storage account name you recorded earlier.

```
csharpTypeCopy
```

```
using Azure;
```

```
using Azure.Identity;
```

```
using Azure.Storage.Queues;
```

```
using Azure.Storage.Queues.Models;
```

```
using System;
```

```
using System.Threading.Tasks;
```

```
// Create a unique name for the queue
```

```
// TODO: Replace the <YOUR-STORAGE-ACCT-NAME> placeholder
```

```
string queueName = "myqueue-" + Guid.NewGuid().ToString();
```

```
string storageAccountName = "<YOUR-STORAGE-ACCT-NAME>";
```

```
// ADD CODE TO CREATE A QUEUE CLIENT AND CREATE A QUEUE
```

```
// ADD CODE TO SEND AND LIST MESSAGES
```

```
// ADD CODE TO UPDATE A MESSAGE AND LIST MESSAGES
```

// ADD CODE TO DELETE MESSAGES AND THE QUEUE

3. Press **ctrl+s** to save your changes.

Add code to create a queue client and create a queue

Now it's time to add code to create the queue storage client and create a queue.

1. Locate the **// ADD CODE TO CREATE A QUEUE CLIENT AND CREATE A QUEUE** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

// Create a DefaultAzureCredentialOptions object to exclude certain credentials

```
DefaultAzureCredentialOptions options = new()
```

```
{
```

```
    ExcludeEnvironmentCredential = true,
```

```
    ExcludeManagedIdentityCredential = true
```

```
};
```

// Instantiate a QueueClient to create and interact with the queue

```
QueueClient queueClient = new QueueClient(
```

```
    new Uri($"https://{storageAccountName}.queue.core.windows.net/{queueName}"),
```

```
    new DefaultAzureCredential(options));
```

```
Console.WriteLine($"Creating queue: {queueName}");
```

// Create the queue

```
await queueClient.CreateAsync();
```

```
Console.WriteLine("Queue created, press Enter to add messages to the queue...");
```

```
Console.ReadLine();
```

2. Press **ctrl+s** to save the file, then continue with the exercise.

Add code to send and list messages in a queue

1. Locate the **// ADD CODE TO SEND AND LIST MESSAGES** comment and add the following code directly after the comment. Be sure to review the code and comments.

```
csharpTypeCopy
```

```
// Send several messages to the queue with the SendMessageAsync method.
```

```
await queueClient.SendMessageAsync("Message 1");
```

```
await queueClient.SendMessageAsync("Message 2");
```

```
// Send a message and save the receipt for later use
```

```
SendReceipt receipt = await queueClient.SendMessageAsync("Message 3");
```

```
Console.WriteLine("Messages added to the queue. Press Enter to peek at the  
messages...");
```

```
Console.ReadLine();
```

```
// Peeking messages lets you view the messages without removing them from the queue.
```

```
foreach (var message in (await queueClient.PeekMessagesAsync(maxMessages:  
10)).Value)
```

```
{
```

```
    Console.WriteLine($"Message: {message.MessageText}");
```

```
}
```

```
Console.WriteLine("\nPress Enter to update a message in the queue...");
```

```
Console.ReadLine();
```

2. Press **ctrl+s** to save the file, then continue with the exercise.

Add code to update a message and list the results

1. Locate the **// ADD CODE TO UPDATE A MESSAGE AND LIST MESSAGES** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

// Update a message with the UpdateMessageAsync method and the saved receipt

```
await queueClient.UpdateMessageAsync(receipt.MessageId, receipt.PopReceipt,
"Message 3 has been updated");
```

```
Console.WriteLine("Message three updated. Press Enter to peek at the messages
again...");
```

```
Console.ReadLine();
```

// Peek messages from the queue to compare updated content

```
foreach (var message in (await queueClient.PeekMessagesAsync(maxMessages:
10)).Value)
```

```
{
```

```
    Console.WriteLine($"Message: {message.MessageText}");
```

```
}
```

```
Console.WriteLine("\nPress Enter to delete messages from the queue...");
```

```
Console.ReadLine();
```

2. Press **ctrl+s** to save the file, then continue with the exercise.

Add code to delete messages and the queue

1. Locate the **// ADD CODE TO DELETE MESSAGES AND THE QUEUE** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

// Delete messages from the queue with the DeleteMessagesAsync method.

```

foreach (var message in (await queueClient.ReceiveMessagesAsync(maxMessages:
10)).Value)
{
    // "Process" the message

    Console.WriteLine($"Deleting message: {message.MessageText}");

    // Let the service know we're finished with the message and it can be safely deleted.
    await queueClient.DeleteMessageAsync(message.MessageId, message.PopReceipt);
}

Console.WriteLine("Messages deleted from the queue.");
Console.WriteLine("\nPress Enter key to delete the queue...");
Console.ReadLine();

// Delete the queue with the DeleteAsync method.
Console.WriteLine($"Deleting queue: {queueClient.Name}");
await queueClient.DeleteAsync();

Console.WriteLine("Done");

```

2. Press **ctrl+s** to save the file, then **ctrl+q** to exit the editor.

Sign into Azure and run the app

1. In the cloud shell command-line pane, enter the following command to sign into Azure.

TypeCopy

```
az login
```

You must sign into Azure - even though the cloud shell session is already authenticated.

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See [Sign into Azure interactively using Azure CLI](#) for details.

2. Run the following command to start the console app. The app will pause many times during execution waiting for you to press any key to continue. This gives you an opportunity to view the messages in the Azure portal.

TypeCopy

dotnet run

3. In the Azure portal, navigate to the Azure Storage account you created.
4. Expand > **Data storage** in the left navigation and select **Queues**.
5. Select the queue the application creates and you can view the sent messages and monitor what the application is doing.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. In your browser navigate to the Azure portal <https://portal.azure.com>; signing in with your Azure credentials if prompted.
2. Navigate to the resource group you created and view the contents of the resources used in this exercise.
3. On the toolbar, select **Delete resource group**.
4. Enter the resource group name and confirm that you want to delete it.

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.